

Backend Systems Assignment

Session: Backend Fundamentals & Architecture

Part 1 – Backend Flow Understanding

Scenario Explanation

In an e-commerce application, when a user logs in, adds products to the cart, places an order, and completes payment, multiple backend processes are triggered. These operations involve APIs, middleware, database interactions, and external services such as payment gateways. Below is the complete backend flow explained step by step.

1. User Login

API Used

POST /api/auth/login

Backend Flow

When the user enters email and password and clicks login:

1. The frontend sends an HTTP POST request to the login API.
2. The request first passes through middleware:
 - **Validation middleware** checks whether required fields are present and correctly formatted.
 - **Logging middleware** records the request details.
3. The backend service checks the user credentials from the database.
4. If credentials are correct, a JWT (JSON Web Token) is generated.
5. The token is returned in the response.
6. The frontend stores this token and includes it in future requests.

HTTP Status Codes

- 200 OK – Login successful.
- 400 Bad Request – Missing or invalid input.
- 401 Unauthorized – Incorrect email or password.
- 500 Internal Server Error – Database or server issue.

2. Add Product to Cart

API Used

POST /api/cart/add

Backend Flow

1. The frontend sends product ID and quantity along with the Authorization token.
2. The request goes through:
 - **Authentication middleware** to verify JWT.
 - **Validation middleware** to validate input data.
3. Backend checks:
 - Whether the product exists.
 - Whether sufficient stock is available.
4. The product is added to the user's cart in the database.
5. Updated cart details are returned in response.

HTTP Status Codes

- 200 OK – Product added successfully.
- 401 Unauthorized – Invalid or expired token.
- 404 Not Found – Product not found.
- 409 Conflict – Insufficient stock.
- 500 Internal Server Error – System failure.

3. Place Order

API Used

POST /api/orders

Backend Flow

1. User initiates order placement.
2. Authentication middleware verifies user identity.
3. Backend retrieves cart details.
4. The system re-validates stock to avoid race conditions.
5. Total order amount is calculated.
6. A new order record is created with status PENDING_PAYMENT.

7. Stock may be temporarily reserved.

HTTP Status Codes

- 201 Created – Order created successfully.
- 400 Bad Request – Cart is empty.
- 409 Conflict – Stock unavailable.
- 500 Internal Server Error – Server failure.

4. Payment Processing

API Used

POST /api/payments

Backend Flow

1. Payment details are sent to the backend.
2. Backend communicates with a third-party payment gateway.
3. Payment gateway processes the transaction.
4. Gateway sends confirmation via webhook.
5. Backend updates order status:
 - SUCCESS if payment is completed.
 - FAILED if payment fails.

HTTP Status Codes

- 200 OK – Payment successful.
- 402 Payment Required – Payment failed.
- 504 Gateway Timeout – Payment gateway delay.
- 500 Internal Server Error – System error.

What Happens if Payment Fails?

If payment fails:

- Order status is updated to FAILED.
- Reserved stock is restored.
- User is notified about failure.
- User may retry payment.

- No order is confirmed unless payment success is verified via payment gateway webhook.

This ensures secure and reliable transaction handling.

Request–Response Model Explanation

The backend follows a stateless request–response model:

1. Client sends HTTP request.
2. Request passes through middleware.
3. Controller processes logic.
4. Service layer handles business rules.
5. Database interaction occurs.
6. Response is sent back with appropriate status code and JSON data.

Each request contains necessary authentication information, making the system scalable.

Part 2 – CORS & Security

Scenario

Frontend: <https://shop-frontend.com>

Backend: <https://api-shop.com>

The browser blocks requests due to CORS policy.

1. Why Does This Error Occur?

Browsers follow the Same-Origin Policy for security reasons. When frontend and backend are hosted on different domains, ports, or protocols, the browser treats them as different origins.

Since the origins differ, the browser blocks the request unless the backend explicitly allows it through CORS headers.

2. Required HTTP Headers

To allow access, backend must return:

Access-Control-Allow-Origin: <https://shop-frontend.com>

Access-Control-Allow-Methods: GET, POST, PUT, DELETE

Access-Control-Allow-Headers: Content-Type, Authorization

Access-Control-Allow-Credentials: true

These headers inform the browser that the request is permitted

3. What is a Preflight Request?

A preflight request is an HTTP OPTIONS request automatically sent by the browser before the actual request.

It checks:

- Whether the HTTP method is allowed.
- Whether custom headers are allowed.
- Whether credentials are supported.

If the backend responds correctly, the actual request proceeds.

4. Why is Access-Control-Allow-Origin: * Risky?

Using * allows all domains to access the API.

Risks:

- Unauthorized websites may access APIs.
- Increased risk of data theft.
- Security vulnerabilities in case of sensitive data.

In production, specific domain whitelisting is recommended.

Part 3 – Architecture Decision

Traffic Expectations

- Normal: 10,000 users per day.
- Festival sale: 5 lakh users.

1. Architecture Choice

I would start with an **N-layered Monolithic Architecture**.

Reasons:

- Easier to develop and manage.
- Suitable for moderate traffic.
- Faster initial deployment.
- Less infrastructure complexity.
- Easier debugging.

Microservices would increase complexity unnecessarily in the early stage.

2. When to Introduce Load Balancer?

A load balancer should be introduced when:

- Traffic increases significantly.
- CPU usage becomes high.
- Multiple backend instances are required.
- High availability is necessary.

Load balancer distributes incoming requests across multiple servers.

3. Preventing Single Point of Failure

To prevent failure:

- Deploy multiple backend instances.
- Use database replication.
- Implement health checks.
- Use cloud-managed services.
- Store sessions in centralized storage like Redis.

4. Horizontal vs Vertical Scaling

Vertical Scaling

- Increasing CPU/RAM of a single server.
- Simple but limited.

Horizontal Scaling

- Adding multiple servers.
- More scalable and fault tolerant.

Best Approach

For this scenario, **Horizontal Scaling** is better because:

- Festival traffic is unpredictable.
- Servers can be added dynamically.
- Provides better availability.

Part 4 – Failure Scenario

Situation

- CPU usage reaches 100%.
- API response time increases.
- Some requests fail.

1. Possible Root Causes

- Unoptimized database queries.
- Missing database indexes.
- Infinite loops in code.
- Sudden traffic spike.
- Memory leak.
- Blocking operations.
- Absence of caching.

2. Immediate Short-Term Fix

- Restart service.
- Increase server resources.
- Add more instances.
- Enable rate limiting.
- Disable heavy background tasks temporarily.

This helps stabilize the system quickly.

3. Long-Term Architectural Solution

- Implement caching (Redis).
- Optimize database queries and indexing.
- Introduce asynchronous processing using message queues.
- Add monitoring tools.
- Configure auto-scaling.
- Use CDN for static resources.

The goal is to design a resilient system capable of handling peak load efficiently.